

When Does Recursion Pay Off?

An Open, \$0 Reproduction and Honest Dissection of
Two-Timescale Meta-Skill Evolution

loversky02
github.com/loversky02/metaskill-evolve

2026

Abstract

Self-improving LLM agents rewrite their own skill files from execution traces. METASKILL-EVOLVE [4] makes this *recursive* with a two-timescale scheme: a fast loop evolves a task skill s , while a slow loop evolves a branch-local meta-skill $m = (\psi, \sigma, \alpha, \pi, \varepsilon)$ that parameterises the five agents of the improvement pipeline itself, under the same pipeline applied to itself. We contribute (i) an open, \$0, Apple-Silicon reproduction of the method with a single frozen backbone, and (ii) an honest dissection of *when* the recursion pays off. Decomposing the original results shows the fast loop is the workhorse and the recursive meta-loop contributes a modest, task-dependent increment (+6.4/+8.1/+1.9 points over single-level evolution on OfficeQA/SealQA/ALFWorld). We hypothesise a *backbone-strength law*: the meta-loop’s marginal value shrinks as the frozen backbone strengthens, making recursive self-improvement chiefly worthwhile for cheap or weak models. We instrument this with a controlled mechanism study and two further probes—English→Vietnamese meta transfer, and the token cost of recursion. We are explicit about scope: absolute numbers require the paper’s 31B backbone, which does not fit our 24 GB budget; our engine reproduces the *method* and the *qualitative structure*, and live confirmation of the backbone-strength law on real backbones is ongoing.

1 Introduction

External skills—reusable procedural knowledge handed to an agent—extend what LLM agents can do on long-horizon tasks. Self-improving agents rewrite these skill files from their own traces. METASKILL-EVOLVE observes that such self-evolution is still *non-recursive*: it improves *what* the agent does (the task skill) but not *how* it improves (the improvement procedure), which is authored once and frozen. METASKILL-EVOLVE closes this gap with a second, slower loop that evolves the improvement procedure with the same machinery.

This paper is not a new method. It is a reproduction and a dissection, asking a single question a practitioner cares about: **given a fixed backbone and a token budget, is the recursion worth it, and when?** Our contributions:

- An open, dependency-light engine (§3) that reproduces the two-timescale method and runs entirely offline at \$0 on a 24 GB Mac.
- A decomposition showing the fast loop dominates and the recursion is a modest, task-dependent bonus (§4.1).
- The backbone-strength law as a testable hypothesis, with a controlled mechanism study (§4.2), plus EN→VI transfer (§4.3) and a cost analysis (§4.4).

2 Background: METASKILL-EVOLVE

Let $U(s) = \mathbb{E}_{(x,y) \sim \mathcal{T}}[r(A_s(x), y)]$ be the utility of a task skill s , where A_s is the agent conditioned on s and $r \in [0, 1]$ scores predictions. The fast loop (Alg. 1) mines a worst case, runs the pipeline Analyzer $\rightarrow$ Retriever $\rightarrow$ Allocator $\rightarrow$ Proposer $\rightarrow$ Evolver to produce K children, scores them, and commits to a branch DAG. Every H iterations the slow loop (Alg. 2) aggregates meta-productivity $P(m | s) = \mathbb{E}[\frac{1}{K} \sum_k (U(s'_k) - U(s))]$, synthesises a meta-failure trace, and applies the *same* pipeline to the five meta files $\{\psi, \sigma, \alpha, \pi, \varepsilon\}$. All five agents share one frozen backbone. Key defaults: $H=2$, $K_{\max}=3$, frontier size 3 with weights (1, 0.5, 0.25).

3 An Open, \$0 Reproduction

Our engine mirrors the method: versioned Markdown skill/meta-skill files with snapshot/restore, a SQLite branch DAG with a weighted frontier, the five agents reading their meta files, and a provider-agnostic backbone (mock for tests, a local MLX model for the weak end, or an OpenAI-compatible gateway for the strong end). The whole pipeline is text-only with a frozen backbone—no gradient updates—so both loops run at \$0. The reference implementation is 33 unit tests green with no API key, GPU, or downloads.

4 Dissection

4.1 The fast loop is the workhorse

Decomposing Table 1 of Wang et al. [4] into the single-level jump (fast loop, meta frozen) and the recursive increment (Table 1) shows that 70–90% of the gain over the raw backbone comes from the fast loop alone; the recursion adds +6.4/ + 8.1/ + 1.9. On ALFWorld the baseline is already 92.3% (near ceiling; static skills even *hurt*), so the +1.9 is within noise and a poor test of recursion.

Table 1: Decomposition of the reported gains. Recursion is the small part.

Benchmark	No-skill	Static	Single-level	METASKILL-EVOLVE	Recursion Δ
OfficeQA	31.78	36.09	48.94	55.32	+6.38
SealQA	29.17	29.41	37.21	45.26	+8.05
ALFWorld	92.31	90.38	92.31	94.23	+1.92

4.2 A backbone-strength law

Hypothesis. The recursive meta-loop rewrites the prompts that scaffold the pipeline agents. A strong backbone already analyses and proposes well, so this scaffolding is redundant; a weak backbone benefits more. We therefore predict the meta-gain *decreases* as the backbone strengthens—so recursive self-improvement is chiefly worthwhile for cheap/weak models, which is also the economically interesting regime.

Mechanism study (toy). We construct a controlled task where an agent’s behaviour reads a directive from its own meta file, so the slow loop can causally improve the pipeline. With proxy backbones of increasing competence, the meta-gain falls monotonically to zero (Table 2). *These are toy numbers that verify the mechanism and the harness, not benchmark results.*

Live probe: both failure modes bracket a narrow regime. We ran the procedure on real backbones—Claude Haiku 4.5 and Opus 4.8 (gateway) and a local Gemma-2-2B (MLX). On short multi-step word problems, every backbone reached *ceiling* before any evolution ($U_0=1.0$): nothing to improve, meta-gain 0. We then tried a genuinely hard benchmark, SealQA Seal-Hard [3], with a bare LLM (no tools); there Haiku scored 0/12 ($U_0=0.0$)—the opposite *floor* failure. The cause is instructive: SealQA is a *knowledge/search* benchmark (fresh, trap-laden fact-seeking), and evolving a text skill on a frozen backbone cannot supply missing knowledge—the paper’s own SealQA gains came from an agent *with* search tools, which our \$0 bare-LLM reproduction deliberately omits. Between these two failure modes lies the narrow regime where recursive skill-evolution actually pays off: tasks whose bottleneck is an *improvable procedure*, not triviality (ceiling) or missing knowledge/tools (floor). *Characterising that regime—rather than assuming recursion always helps—is the honest empirical contribution here.* Table 3 records both live nulls.

Table 2: Mechanism study on a toy with proxy backbones (offline). Meta-gain vanishes as the backbone strengthens.

Backbone (proxy)	Single-level	Two-level	Meta-gain
weak	0.50	1.00	+0.50
mid	0.75	1.00	+0.25
strong	1.00	1.00	+0.00

Table 3: Live probe: both failure modes. On easy word problems the base policy is already at ceiling ($U_0=1.0$); on knowledge-hard SealQA (no tools) it is at the floor ($U_0=0.0$). Either way the recursion has no headroom, so the meta-gain is 0 — the law needs a procedure-bottlenecked task in between, not stronger claims from this data.

Task	Backbone	U_0	Single	Two-level	Meta-gain
Word problems	Gemma-2-2B (MLX)	1.00	1.00	1.00	+0.00 (ceiling)
Word problems	Claude Haiku 4.5	1.00	1.00	1.00	+0.00 (ceiling)
SealQA Seal-Hard	Claude Haiku 4.5 (no tools)	0.00		floor – not run	

4.3 English→Vietnamese meta transfer

Meta-skills are language-agnostic improvement policies. We ask whether a meta-skill evolved on English tasks accelerates task-skill evolution on Vietnamese ones (GSM8K $\rightarrow$ vi-gsm8k). The harness runs; live numbers are TODO.

4.4 The cost of recursion

Recursion multiplies inference—you evolve the evolvers. We instrument calls and tokens and compare fast-only vs. two-level at equal iteration budget, reporting utility gained per token. On the toy the recursion costs roughly +30% tokens for its extra gain; the live token/accuracy trade-off is TODO.

5 Related Work

Skill-augmented and self-evolving agents are a fast-growing cluster [2, 1]. METASKILL-EVOLVE is distinctive in making the improvement procedure itself recursive with no extra model. Our work is orthogonal: we do not propose a new evolver but dissect *when* recursion helps, on a frozen backbone, at \$0.

6 Limitations and Honesty

(1) The paper’s absolute numbers used a 31B backbone (20 GB at 4-bit) that does not fit our 24 GB budget; we reproduce the method and qualitative structure, not the exact numbers. (2) The backbone-strength law is currently supported only by a constructed mechanism study; the proxy backbones were designed to express the predicted shape, so they demonstrate the harness, not the law. (3) OfficeQA data is access-gated; SealQA and ALFWorld are public. Live experiments are the immediate next step.

7 Conclusion

Two-timescale recursion is an elegant idea, but its benefit is modest and task-dependent, concentrated where the improvement pipeline—not the base policy—is the bottleneck. We predict, and are testing, that this benefit is a decreasing function of backbone strength. An open \$0 engine makes the question cheap for anyone to probe.

References

- [1] Anonymous. Evolving-rl: End-to-end optimization of experience-driven self-evolving capability within agents. *arXiv preprint arXiv:2605.10663*, 2026.
- [2] Anonymous. SkillRL: Evolving agents via recursive skill-augmented reinforcement learning. *arXiv preprint arXiv:2602.08234*, 2026.
- [3] Thinh Pham et al. SealQA: Raising the bar for reasoning in search-augmented language models. *arXiv preprint arXiv:2506.01062*, 2025.
- [4] Zefeng Wang, Minxi Yan, Jinhe Bi, Sikuan Yan, Volker Tresp, and Yunpu Ma. MetaSkill-Evolve: Recursive self-improvement of LLM agents via two-timescale meta-skill evolution. *arXiv preprint arXiv:2607.05297*, 2026.